



THE PERIPHERALS

USB | UART | I2C | SPI | PWM | Timer | Watchdog | The Realtime Clock (RTC) | ADC | SSI

THE PROGRAMMABLE INPUT/OUTPUT

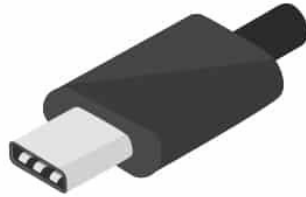
USB	Compare the registers used in USB	Ap
UART	How to transfer data using UART?	An
I2C	How to perform transformation using I2C?	An
SPI	Analyze the operation of SPI	An
PWM	Experiment with PWM in RP2040	An
Timer	Demonstrate timer and related operations using RP2040	Ap
Watchdog	Justify the sequence of operations in functioning a watchdog timer	Ap
The Realtime Clock (RTC)	Outline RTC	An
ADC SSI	Summarize ADC and SSI features in RP2040	Ap



USB TYPE A



USB TYPE B



USB TYPE C



USB MINI A



USB MINI B



USB MICRO C



USB MICRO C



USB MICRO B SUBER SPEED

US

ILO

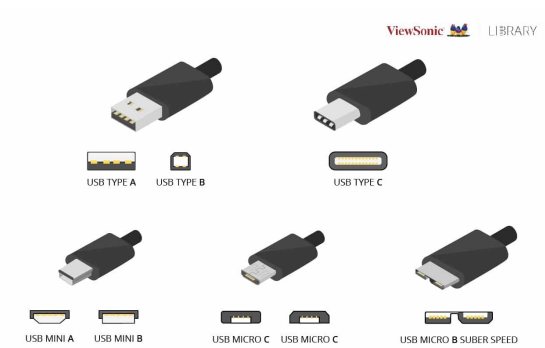
Compare the registers
used in USB

USB

Device Mode: This means the microcontroller can act like a USB device when you connect it to your computer. It can do this at "Full Speed, which is a data transfer rate of 12 megabits per second (Mbps).

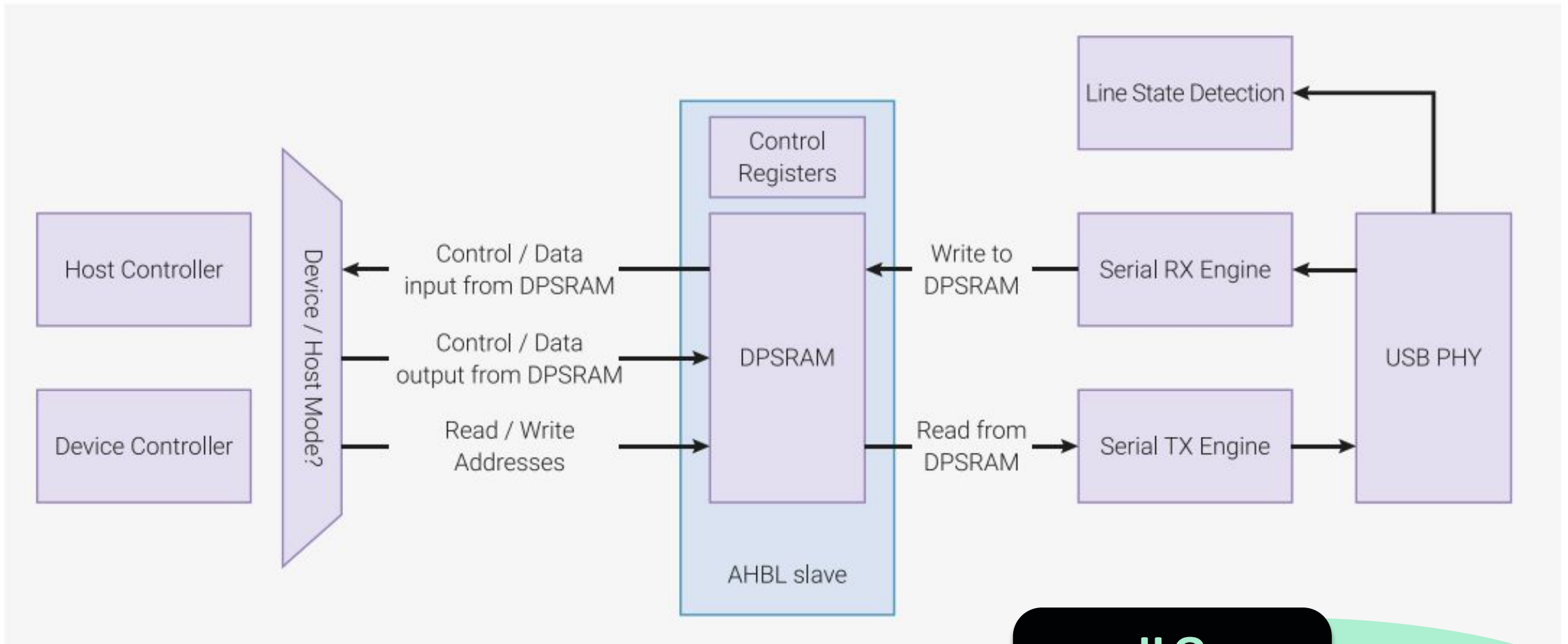
Host Mode: In this mode, the RP2040 can communicate with other USB devices, both those that work at Full Speed (12Mbps) and those that work at Low Speed (1.5Mbps). It can also talk to multiple devices through a USB hub.

For the RP2040 to do all of this, there's a part of it called the **USB controller**. The USB controller also has a place to store data called **DPSRAM (4kB)**



ILO

Compare the registers
used in USB



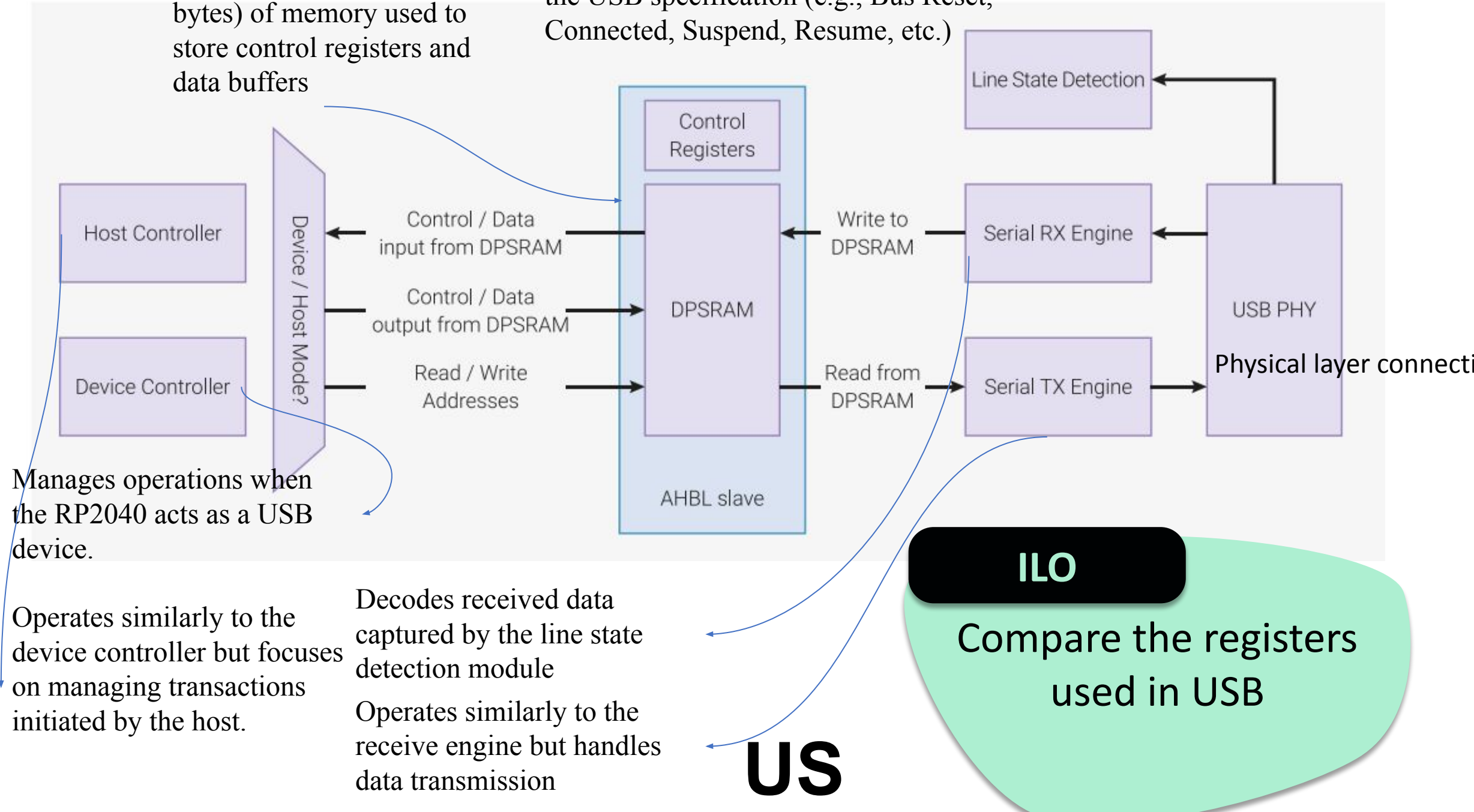
ILO

Compare the registers
used in USB

US

Provides 4KB (4096 bytes) of memory used to store control registers and data buffers

Detects various line states defined by the USB specification (e.g., Bus Reset, Connected, Suspend, Resume, etc.)



Manages operations when the RP2040 acts as a USB device.

Operates similarly to the device controller but focuses on managing transactions initiated by the host.

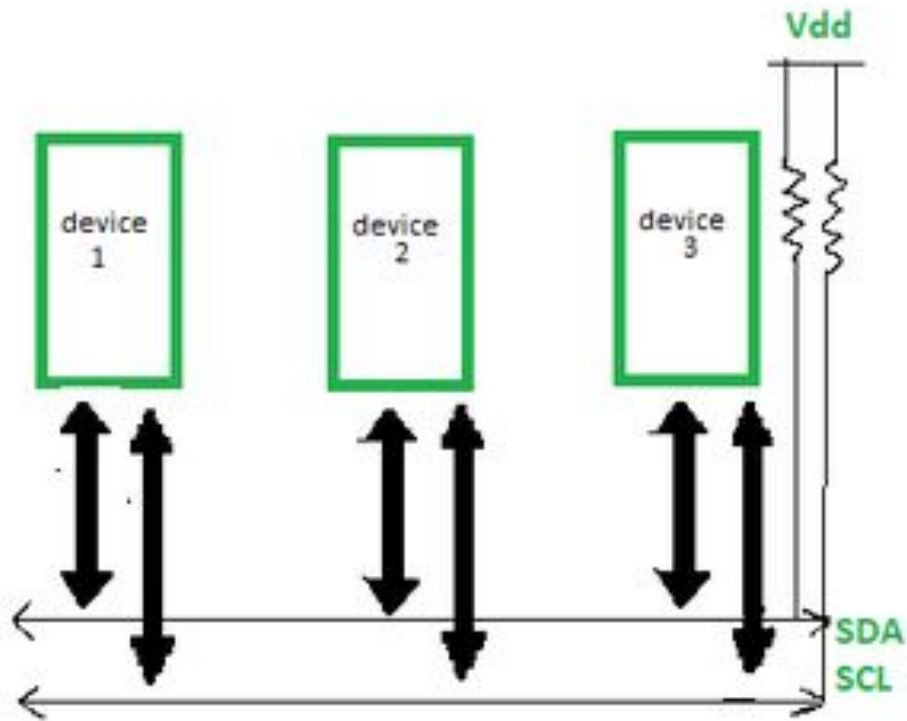
Decodes received data captured by the line state detection module

Operates similarly to the receive engine but handles data transmission

ILO

Compare the registers used in USB

US



I2C

ILO

How to perform
transformation
using I2C?

<https://www.geeksforgeeks.org/i2c-communication-protocol/>

I2C

ILO

How to perform
transformation
using I2C?

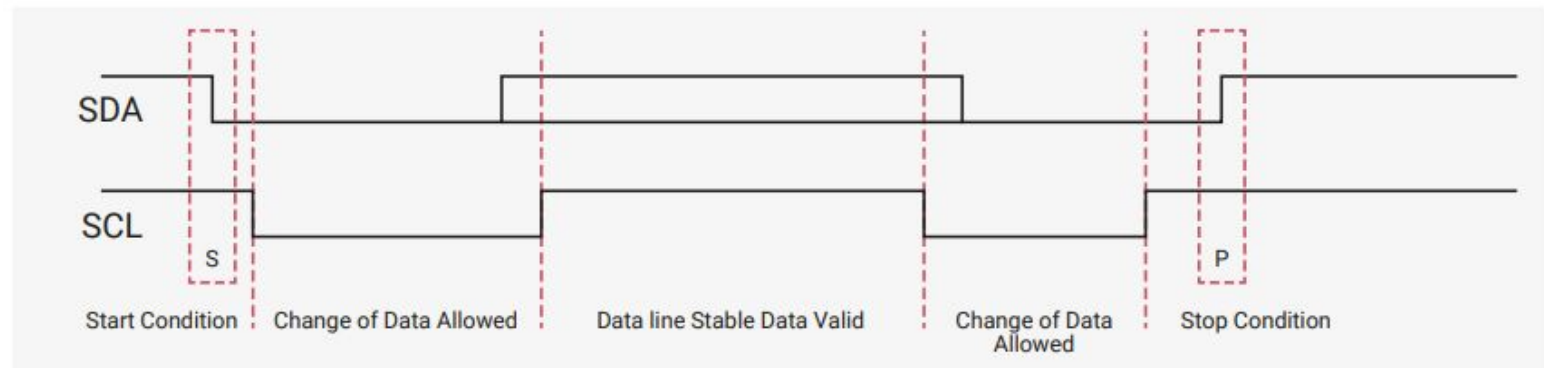
I2C is a commonly used 2-wire interface that can be used to connect devices for low speed data transfer using clock SCL and data SDA wires.

I2C Terminology

- Transmitter
- Receiver
- Master
- Slave
- Multi-master
- Arbitration
- Synchronization

I2C Protocols

START and STOP Conditions

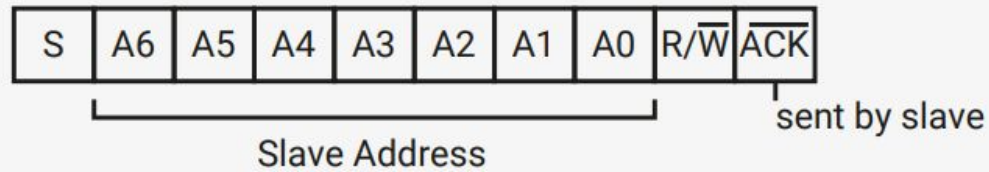


I2C

ILO

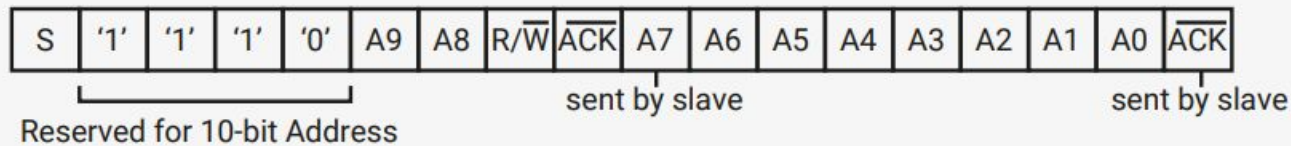
How to perform
transformation
using I2C?

Addressing Slave Protocol



S = START Condition $\overline{ACK}$ = Acknowledge R/ $\overline{W}$ = Read/Write Pulse

7-bit Address Format



S = START Condition $\overline{ACK}$ = Acknowledge R/ $\overline{W}$ = Read/Write Pulse

10-bit Address Format

I2C

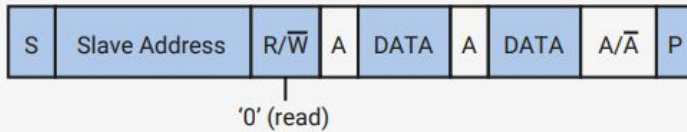
Transmitting and Receiving Protocol

ILO

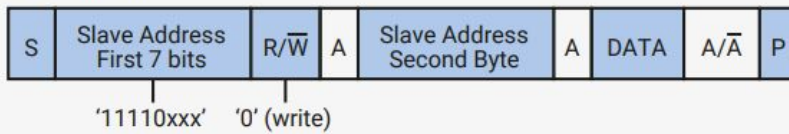
How to perform
transformation
using I2C?

Master-Transmitter and Slave-Receiver

For 7-bit Address



For 10-bit Address



■ From Master to Slave

A = Acknowledge (SDA low)

S = START Condition

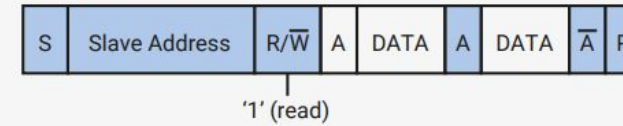
□ From Slave to Master

Ā = No Acknowledge (SDA high)

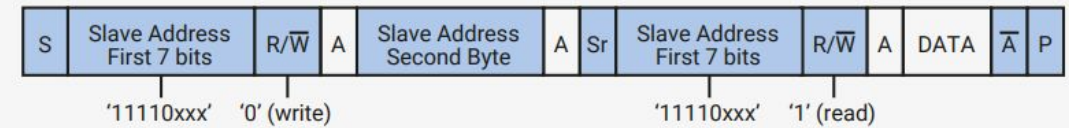
P = STOP Condition

Master-Receiver and Slave-Transmitter

For 7-bit Address



For 10-bit Address



■ From Master to Slave

A = Acknowledge (SDA low)

R = RESTART Condition

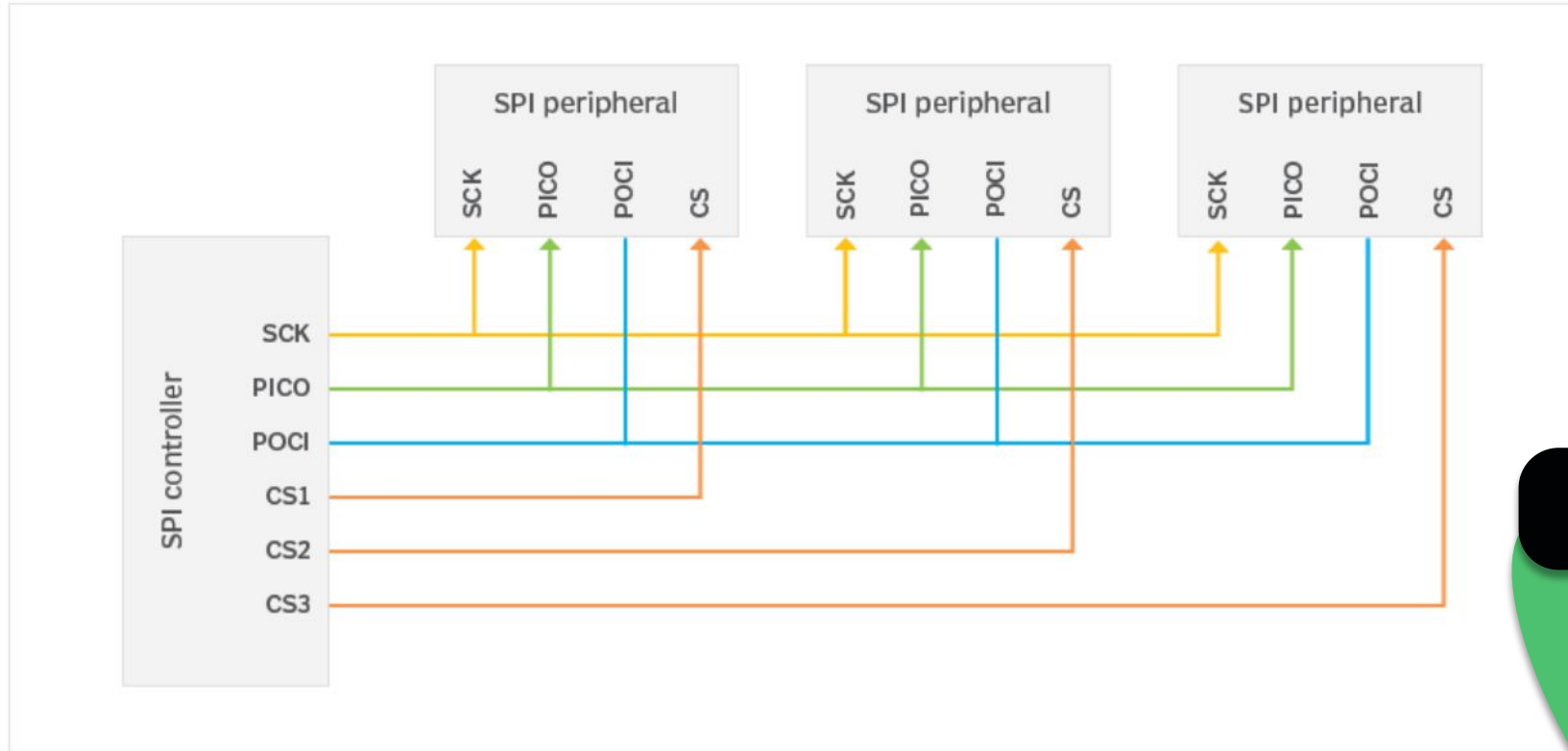
□ From Slave to Master

Ā = No Acknowledge (SDA high)

P = STOP Condition

S = START Condition

Standard SPI



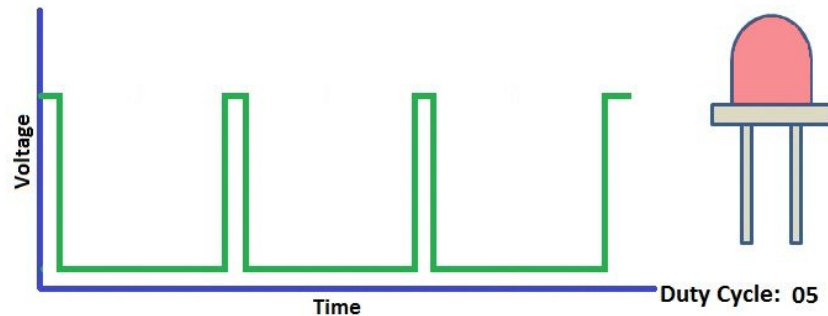
SPI

ILO

Analyze the operation
of SPI

<https://www.techtarget.com/whatis/definition/serial-peripheral-interface-SPI>

Pulse Width Modulation (PWM) is a technique used to control the amount of power delivered to an electrical device by rapidly turning it on and off. This is done by varying the width of the on-time while keeping the total time period constant. In simple terms, PWM is like flicking a light switch on and off very quickly to control the brightness of a light.



**PW
M**

ILO

**Experiment with PWM in
RP2040**

Pulse width modulation (PWM) is a scheme where a digital signal provides a smoothly varying average voltage. This is achieved with *positive pulses* of some *controlled width*, at *regular intervals*. The fraction of time spent high is known as the **duty cycle**.

PWM In RP2040

ILO

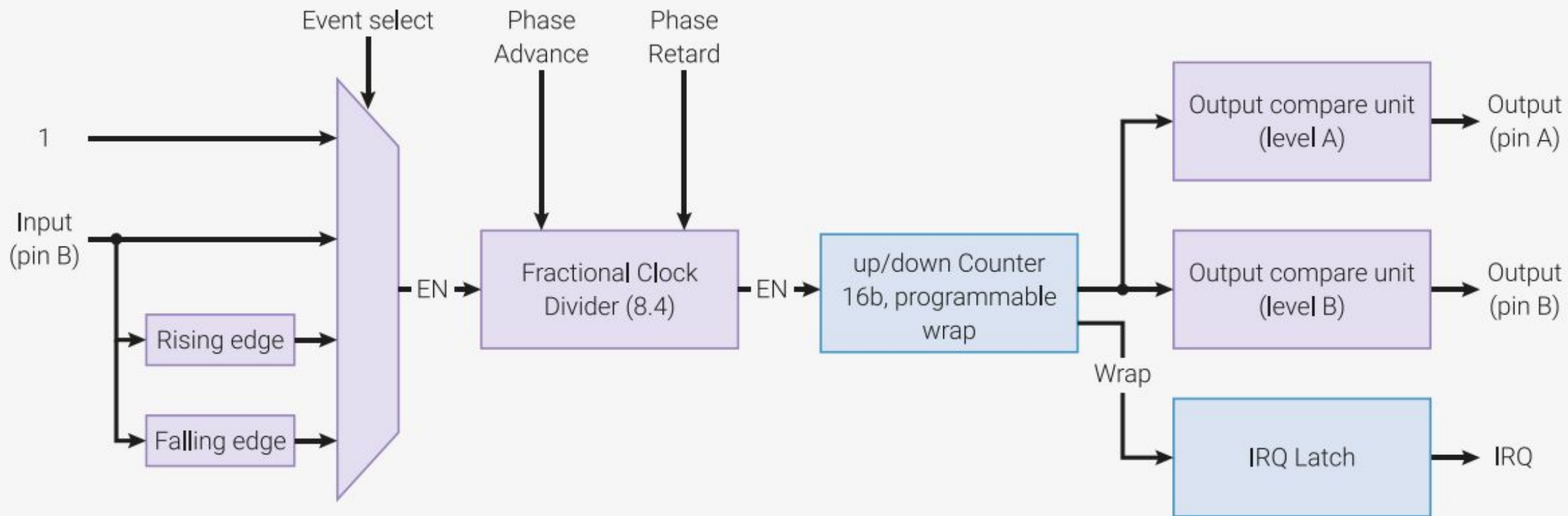
Experiment with PWM in
RP2040

The RP2040 PWM block has **8 identical slices**. Each slice can drive **two PWM output** signals, or measure the frequency or duty cycle of an input signal. This gives a total of up to **16 controllable PWM outputs**. All 30 GPIO pins can be driven by the PWM block.

PWM In RP2040

ILO

Experiment with PWM in
RP2040



Features of PWM in RP2040

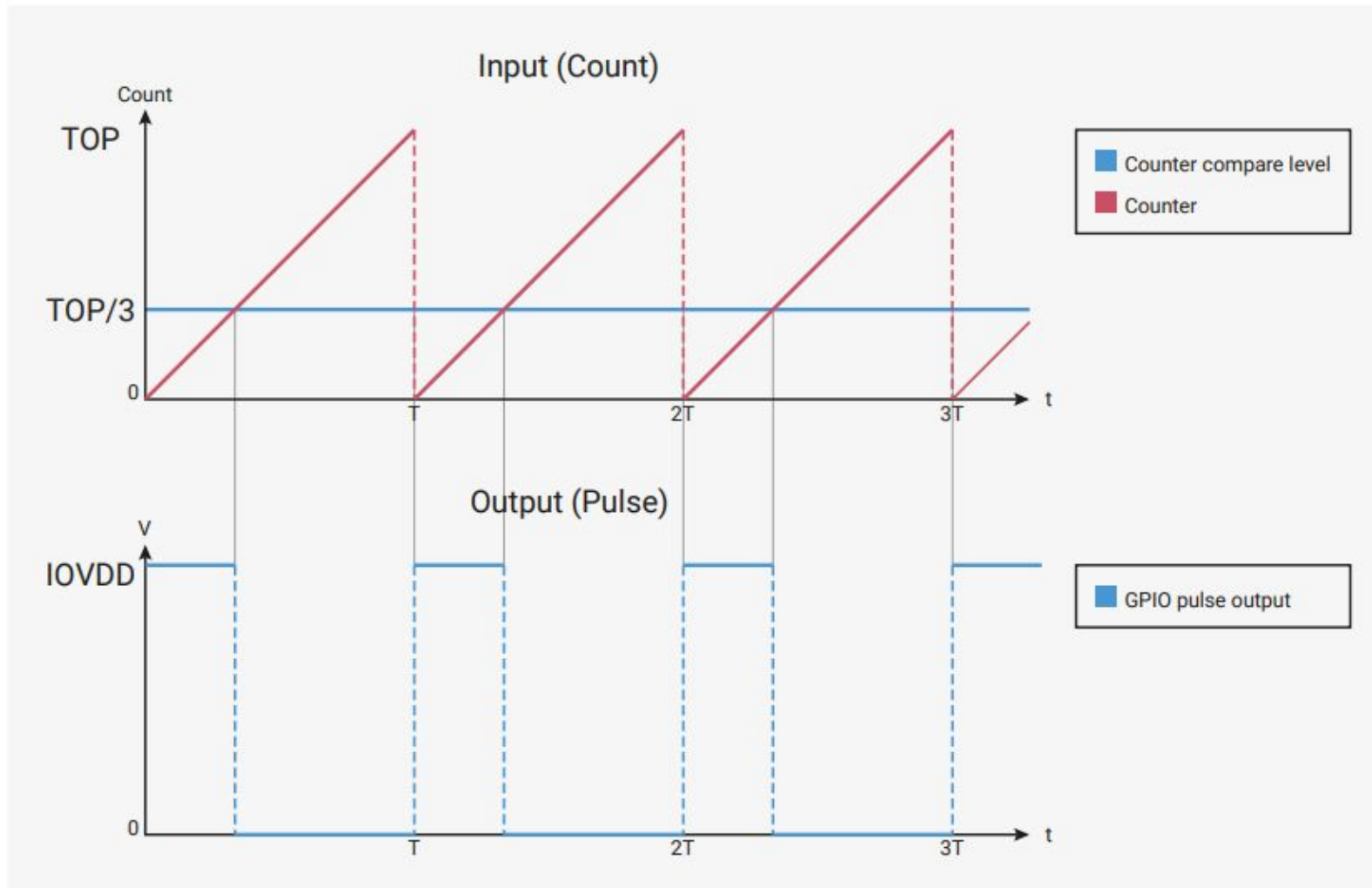
- 16-bit counter
- 8.4 fractional clock divider
- Two independent output channels, duty cycle from 0% to 100% inclusive
- Dual slope and trailing edge modulation
- Edge-sensitive input mode for frequency measurement
- Level-sensitive input mode for duty cycle measurement
- Configurable counter wrap value
- Interrupt request and DMA request on counter wrap
- Phase can be precisely advanced or retarded while running (increments of one count)

All 30 GPIO pins on RP2040 can be used for PWM:

GPIO	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
PWM Channel	0A	0B	1A	1B	2A	2B	3A	3B	4A	4B	5A	5B	6A	6B	7A	7B
GPIO	16	17	18	19	20	21	22	23	24	25	26	27	28	29		
PWM Channel	0A	0B	1A	1B	2A	2B	3A	3B	4A	4B	5A	5B	6A	6B		

- The 16 PWM channels (8 2-channel slices) appear on GPIO0 to GPIO15, in the order PWM0 A, PWM0 B, PWM1 A...
- This repeats for GPIO16 to GPIO29. GPIO16 is PWM0 A, GPIO17 is PWM0 B, so on up to PWM6 B on GPIO29
- The same PWM output can be selected on two GPIO pins; the same signal will appear on each GPIO.
- If a PWM B pin is used as an input, and is selected on multiple GPIO pins, then the PWM slice will see the logical OR of those two GPIO inputs

Pulse Width Modulation



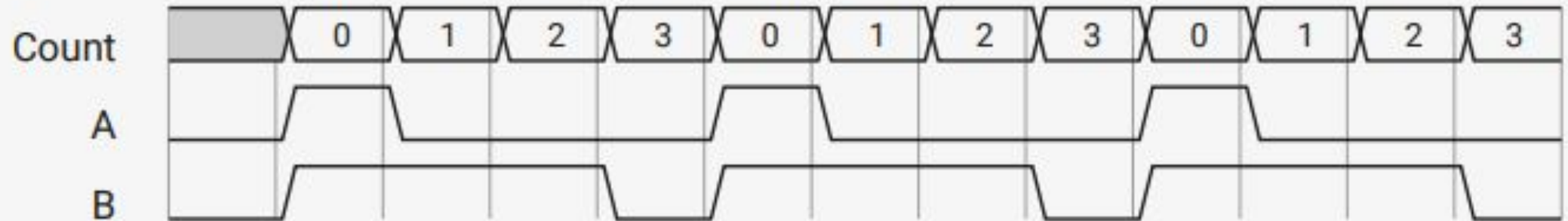
The counting period is controlled by the TOP (is the wrapping value) register, with a maximum possible period of 65536 cycles, as the counter and TOP are 16 bits in size. The input values are configured via the CC register.

Pulse Width Modulation

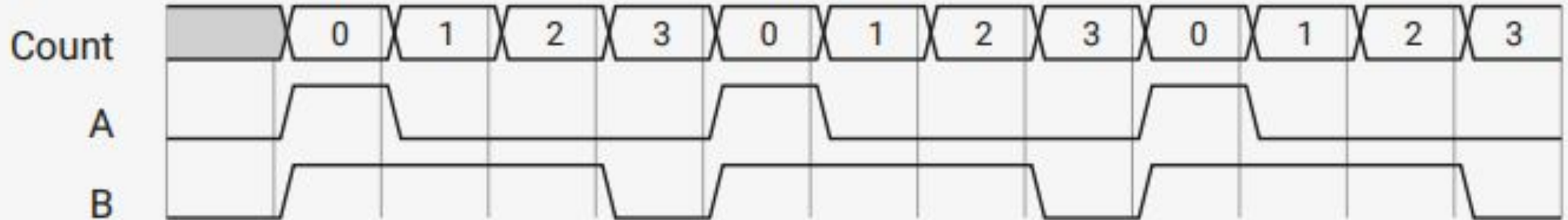
```
14  // Tell GPIO 0 and 1 they are allocated to the PWM
15  gpio_set_function(0, GPIO_FUNC_PWM);
16  gpio_set_function(1, GPIO_FUNC_PWM);
17
18  // Find out which PWM slice is connected to GPIO 0 (it's slice 0)
19  uint slice_num = pwm_gpio_to_slice_num(0);
20
21  // Set period of 4 cycles (0 to 3 inclusive)
22  pwm_set_wrap(slice_num, 3);
23  // Set channel A output high for one cycle before dropping
24  pwm_set_chan_level(slice_num, PWM_CHAN_A, 1);
25  // Set initial B output high for three cycles before dropping
26  pwm_set_chan_level(slice_num, PWM_CHAN_B, 3);
27  // Set the PWM running
28  pwm_set_enabled(slice_num, true);
```



```
22  pwm_set_wrap(slice_num, 3);  
23  // Set channel A output high for one cycle before dropping  
24  pwm_set_chan_level(slice_num, PWM_CHAN_A, 1);  
25  // Set initial B output high for three cycles before dropping  
26  pwm_set_chan_level(slice_num, PWM_CHAN_B, 3);  
27  // Set the PWM running  
28  pwm_set_enabled(slice_num, true);
```




```
22  pwm_set_wrap(slice_num, 3);  
23  // Set channel A output high for one cycle before dropping  
24  pwm_set_chan_level(slice_num, PWM_CHAN_A, 1);  
25  // Set initial B output high for three cycles before dropping  
26  pwm_set_chan_level(slice_num, PWM_CHAN_B, 3);  
27  // Set the PWM running  
28  pwm_set_enabled(slice_num, true);
```



Configuring PWM Period

When free-running, the period of a PWM slice's output (measured in system clock cycles) is controlled by three parameters:

- The TOP register
- Whether phase-correct mode is enabled (CSR_PH_CORRECT)
- The DIV register

Unless told otherwise,
the DIV_FRAC is 15

$$f_{PWM} = \frac{f_{sys}}{\text{period}} = \frac{f_{sys}}{(\text{TOP} + 1) \times (\text{CSR_PH_CORRECT} + 1) \times \left(\text{DIV_INT} + \frac{\text{DIV_FRAC}}{16} \right)}$$

- **The TOP register**
- Whether phase-correct mode is enabled (CSR_PH_CORRECT)
- **The DIV register**

The PWM registers start at a base address of 0x40050000 (defined as PWM_BASE in SDK).

Offset	Name	Info
0x00	CH0_CSR	Control and status register
0x04	CH0_DIV	INT and FRAC form a fixed-point fractional number. Counting rate is system clock frequency divided by this number. Fractional division uses simple 1st-order sigma-delta.
0x08	CH0_CTR	Direct access to the PWM counter
0x0c	CH0_CC	Counter compare values
0x10	CH0_TOP	Counter wrap value
...	...	...

PWM: CH0_TOP, CH1_TOP, ..., CH6_TOP, CH7_TOP Registers

Offsets: 0x10, 0x24, ..., 0x88, 0x9c

Bits	Description	Type	Reset
31:16	Reserved.	-	-
15:0	Counter wrap value	RW	0xffff

- The TOP register
- **Whether phase-correct mode is enabled (CSR_PH_CORRECT)**
- The DIV register

PWM: CH0_CSR, CH1_CSR, ..., CH6_CSR, CH7_CSR Registers

Offsets: 0x00, 0x14, ..., 0x78, 0x8c

Description

Control and status register

Bits	Name	Description	Type	Reset
31:8	Reserved.	-	-	-
7	PH_ADV	Advance the phase of the counter by 1 count, while it is running. Self-clearing. Write a 1, and poll until low. Counter must be running at less than full speed ($\text{div_int} + \text{div_frac} / 16 > 1$)	SC	0x0
6	PH_RET	Retard the phase of the counter by 1 count, while it is running. Self-clearing. Write a 1, and poll until low. Counter must be running.	SC	0x0
5:4	DIVMODE	0x0 → Free-running counting at rate dictated by fractional divider 0x1 → Fractional divider operation is gated by the PWM B pin. 0x2 → Counter advances with each rising edge of the PWM B pin. 0x3 → Counter advances with each falling edge of the PWM B pin.	RW	0x0
3	B_INV	Invert output B	RW	0x0
2	A_INV	Invert output A	RW	0x0
1	PH_CORRECT	1: Enable phase-correct modulation. 0: Trailing-edge	RW	0x0
0	EN	Enable the PWM channel.	RW	0x0

- The TOP register
- Whether phase-correct mode is enabled (CSR_PH_CORRECT)
- **The DIV register**

PWM: CH0_DIV, CH1_DIV, ..., CH6_DIV, CH7_DIV Registers

Offsets: 0x04, 0x18, ..., 0x7c, 0x90

Description

INT and FRAC form a fixed-point fractional number.

Counting rate is system clock frequency divided by this number.

Fractional division uses simple 1st-order sigma-delta.

Bits	Name	Description	Type	Reset
31:12	Reserved.	-	-	-
11:4	INT		RW	0x01
3:0	FRAC		RW	0x0

- The TOP register – is the wrapping value
- Whether phase-correct mode is enabled (CSR_PH_CORRECT)
- The DIV register

I. TOP register – 1100 1100 0010 0100

1100 1100 0010 0100 – decimal –

$$1 + 0 + 0 + 8 + 0 + 0 + 64 + 128 + 256 + 0 + 0 + 0 + 4096 = 4653$$

II. Phase correct mode – enabled

III. DIV register - xxxx xxxx xxxx xxxx xxxx 0000 0000 0010 - 2

$$f_{PWM} = \frac{f_{sys}}{\text{period}} = \frac{f_{sys}}{(\text{TOP} + 1) \times (\text{CSR_PH_CORRECT} + 1) \times \left(\text{DIV_INT} + \frac{\text{DIV_FRAC}}{16} \right)}$$

$$f_{PWM} = 52\text{KHz}$$

Time r

ILO

Demonstrate timer and
related operations using
RP2040

In the RP2040 microcontroller, a timer is a built-in feature that helps keep track of time or measure time intervals. It works like a digital stopwatch, and it's handy for various tasks.

Think of it as a small, electronic clock inside the microcontroller.

- 1. Timekeeper:** A timer in RP2040 is like a digital timekeeper. It counts seconds, milliseconds, or even smaller units of time, depending on how it's configured.
- 2. Precise Timing:** It's really good at keeping time accurately. You can use it to measure how long things take to happen, like how fast a motor spins or how long a button is pressed.
- 3. Control:** You can tell the timer to start counting when you want and stop when you want. This gives you control over when and how you measure time.
- 4. Useful for Many Things:** Timers are used in all sorts of projects. They're great for tasks like flashing LEDs at specific intervals, controlling motors, or even making simple games.

Timer - features

ILO

Demonstrate timer and
related operations using
RP2040

- A single 64-bit counter, incrementing once per microsecond
- This counter can be read from a pair of latching registers, for race-free reads over a 32-bit bus.
- Four alarms: match on the lower 32 bits of counter, IRQ on match.

In the RP2040 microcontroller, the timer is a 64-bit counter, but the data bus is only 32 bits wide.

To work with time values, there are pairs of registers:

- TIMEHW and TIMELW are used for writing time. (32 bit)
- TIMEHR and TIMELR are used for reading time. (32 bit)
 - When reading time, accessing the TIMELW register latches the value in the TIMEHR register, ensuring accurate time reading.
 - Alternatively, you can use TIMERAWH and TIMERAWL to read the raw time without latching.

The timer has four alarms, each triggering a separate interrupt. These alarms match on the lower 32 bits of the 64-bit counter, allowing them to be fired up to approximately 4295 seconds (or around 72 minutes) into the future.

To enable an alarm:

1. Enable the interrupt at the timer by setting the appropriate alarm bit in INTE (e.g., $(1 \ll 0)$ for ALARM0).
2. Enable the corresponding timer interrupt at the processor.
3. Write the desired alarm time in microseconds to ALARM0, setting the ARMED bit as a side effect.

Once the alarm fires, the ARMED bit will be cleared. To clear the latched interrupt, write a 1 to the appropriate bit in INTR. These timers are designed for short sleeps; for longer alarms, refer to another section.

4.6.5. List of Registers

The Timer registers start at a base address of `0x40054000` (defined as `TIMER_BASE` in SDK).

Offset	Name	Info
0x00	<code>TIMEHW</code>	Write to bits 63:32 of time always write timelw before timehw
0x04	<code>TIMELW</code>	Write to bits 31:0 of time writes do not get copied to time until timehw is written
0x08	<code>TIMEHR</code>	Read from bits 63:32 of time always read timelr before timehr
0x0c	<code>TIMELR</code>	Read from bits 31:0 of time
0x10	<code>ALARM0</code>	Arm alarm 0, and configure the time it will fire. Once armed, the alarm fires when <code>TIMER_ALARM0 == TIMELR</code> . The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.
0x14	<code>ALARM1</code>	Arm alarm 1, and configure the time it will fire. Once armed, the alarm fires when <code>TIMER_ALARM1 == TIMELR</code> . The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.
0x18	<code>ALARM2</code>	Arm alarm 2, and configure the time it will fire. Once armed, the alarm fires when <code>TIMER_ALARM2 == TIMELR</code> . The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.

4.6.5. List of Registers

The Timer registers start at a base address of `0x40054000` (defined as `TIMER_BASE` in SDK).

Offset	Name	Info
0x00	<code>TIMEHW</code>	Write to bits 63:32 of time always write timelw before timehw
0x04	<code>TIMELW</code>	Write to bits 31:0 of time writes do not get copied to time until timehw is written
0x08	<code>TIMEHR</code>	Read from bits 63:32 of time always read timelr before timehr
0x0c	<code>TIMELR</code>	Read from bits 31:0 of time
0x10	<code>ALARM0</code>	Arm alarm 0, and configure the time it will fire. Once armed, the alarm fires when <code>TIMER_ALARM0 == TIMELR</code> . The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.
0x14	<code>ALARM1</code>	Arm alarm 1, and configure the time it will fire. Once armed, the alarm fires when <code>TIMER_ALARM1 == TIMELR</code> . The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.
0x18	<code>ALARM2</code>	Arm alarm 2, and configure the time it will fire. Once armed, the alarm fires when <code>TIMER_ALARM2 == TIMELR</code> . The alarm will disarm itself once it fires, and can be disarmed early using the ARMED status register.

Memorize this registers address and names

0x24	TIMERAWH	Raw read from bits 63:32 of time (no side effects)
0x28	TIMERAWL	Raw read from bits 31:0 of time (no side effects)
0x2c	DBGPAUSE	Set bits high to enable pause when the corresponding debug ports are active
0x30	PAUSE	Set high to pause the timer
0x34	INTR	Raw Interrupts
0x38	INTE	Interrupt Enable
0x3c	INTF	Interrupt Force
0x40	INTS	Interrupt status after masking & forcing

- I. In the address :0x40054038, Value : 0x00000003
- II. In the address : 0x40054004, Value : 0x00215E80
- III. In the address: 0x40054018, Value is constantly checked

- I. In the address :0x40054038, Value : 0x00000003
- II. **In the address : 0x40054004, Value : 0x00215E80**
- III. In the address: 0x40054018, Value is constantly checked

0x00215E80

$$0x0: 0 * 16^0 = 0 * 1 = 0$$

$$0x8: 8 * 16^1 = 8 * 16 = 128$$

$$0xE: 14 * 16^2 = 14 * 256 = 3584$$

$$0x5: 5 * 16^3 = 5 * 4096 = 20480$$

$$0x1: 1 * 16^4 = 1 * 65536 = 65536$$

$$0x2: 2 * 16^5 = 2 * 1048576 = 2097152$$

2172868 Microseconds

2172868 Microseconds / 1000 = 2172.868 seconds

2172.868/60 = 36.21 Minutes

Watchdog

ILO

Demonstrate timer and related operations using RP2040

In simple terms, the watchdog in the RP2040 microcontroller is like a timer that counts down, and if it ever reaches zero, it can restart parts of the chip. Its primary purpose is to prevent software issues, like getting stuck in a loop, from causing the chip to stop working properly.

Here's what you need to know:

1. **Countdown Timer:** The watchdog is a timer that continuously counts down.
2. **Preventing Stuck Software:** To prevent issues where software gets stuck, a programmer must regularly "pet" the watchdog by writing a value to it. This action resets the countdown timer.

Watchdog

ILO

Demonstrate timer and related operations using RP2040

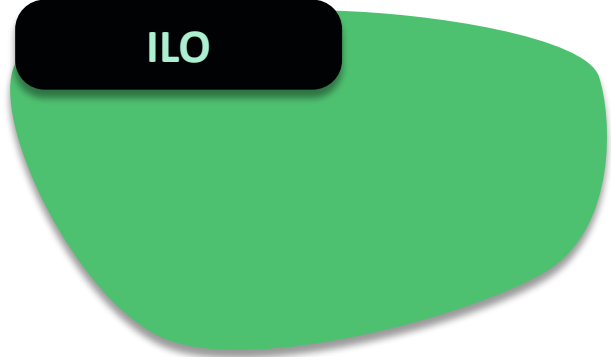
3. Watchdog Reset: The watchdog timer is also connected to the power-on state machine and reset controller. When it's reset, it can affect other parts of the chip, depending on how it's configured.

4. Tick Generation: The watchdog uses a reference clock (`clk_tick`) that's ideally derived from a crystal oscillator for accuracy. This reference clock is divided internally to create a tick (approximately 1 microsecond) used as the watchdog's time unit.

5. Watchdog Counter: The watchdog's countdown value is loaded from the `LOAD` register. You can check the current value in the `CTRL.TIME` register.

6. Scratch Registers: The watchdog includes eight 32-bit scratch registers that store information between resets. These registers can be used to carry data from one chip reset to another.

Watchdog



ILO

7. Bootrom Magic Number: The bootrom checks the watchdog's scratch registers for a "magic number" during boot. If it finds this magic number, it can initiate a soft reset and execute user-specified code.

In essence, the watchdog is a safety feature that ensures the microcontroller keeps running reliably by monitoring and preventing software-related issues. It's a fail-safe mechanism to maintain chip functionality.

Watchdog

ILO

Demonstrate timer and
related operations using
RP2040

WATCHDOG: LOAD Register

Offset: 0x04

Bits	Description	Type	Reset
31:24	Reserved.	-	-
23:0	Load the watchdog timer. The maximum setting is 0xffffffff which corresponds to 0xffffffff / 2 ticks before triggering a watchdog reset (see errata RP2040-E1).	WF	0x000000

RTC

The Real-time Clock (RTC) provides time in human-readable format and can be used to generate interrupts at specific times.

Date/Time Field	Size	Legal values
Year	12 bits	0..4095
Month	4 bits	1..12
Day	5 bits	1..[28,29,30,31], depending on the month
Day of Week	3 bits	0..6. Sunday = 0
Hour	5 bits	0..23
Minute	6 bits	0..59
Seconds	6 bits	0..59

ADC

The ILO logo consists of a black rounded rectangle with the letters 'ILO' in white, positioned above a larger, light green rounded shape that tapers to the right.

RP2040 has an internal analogue-digital converter (ADC) SAR ADC 12 bit

- 1. Number of Channels:** The RP2040 ADC module has 3 channels, labeled ADC0, ADC1, and ADC2. This means you can simultaneously measure up to three different analog signals.
- 2. Resolution:** The ADC provides 12-bit resolution, which means it can represent analog voltages as digital values ranging from 0 to 4095.
- 3. Voltage Reference:** The ADC can be configured to use an internal voltage reference (V_{ref}) or an external reference voltage for conversions. The reference voltage is used as a basis for converting the analog signal to a digital value.
- 4. Conversion Modes:** The RP2040 ADC supports both single-ended and differential conversion modes. In single-ended mode, each channel is measured with respect to the reference voltage. In differential mode, the voltage difference between two channels is measured.
- 5. Sampling Rate:** The ADC can operate at different sampling rates, allowing you to adjust the speed of your analog-to-digital conversions based on your application's requirements.
- 6. Software Control:** The ADC module can be configured and controlled through software using the RP2040 SDK or other programming environments. You can initiate conversions, read the results, and set various configuration options as needed.
- 7. Interrupts:** The ADC can generate interrupts upon completing conversions, making it useful for applications that require real-time data acquisition or monitoring.